

Week 3 — Functions, Lambda, Files, & Exceptions

MIT College Kathmandu — 4-Week Python Bootcamp

Session 1: Functions | Session 2: Files &
Errors

| WHAT WE'RE BUILDING THIS WEEK

- **Session 1:** Mastering reusable logic via Functions (Anatomy, Scope, and Arguments).
- **Session 2:** Advanced transformations (Lambda) and safe data handling (File I/O, Exceptions).
- **Projects:** E-Commerce Cart Engine & Server Log Analyzer.
- **IMS Kickoff:** Briefing on the Inventory & Invoice Management System.

⚠ **Session 2 is the most compressed session of the bootcamp. Stay sharp!**



| LEARNING OBJECTIVES



Functional Mastery

Write clean functions with default params, *args, and **kwargs.



Data Persistence

Read and write files safely using Python context managers.



Error Resilience

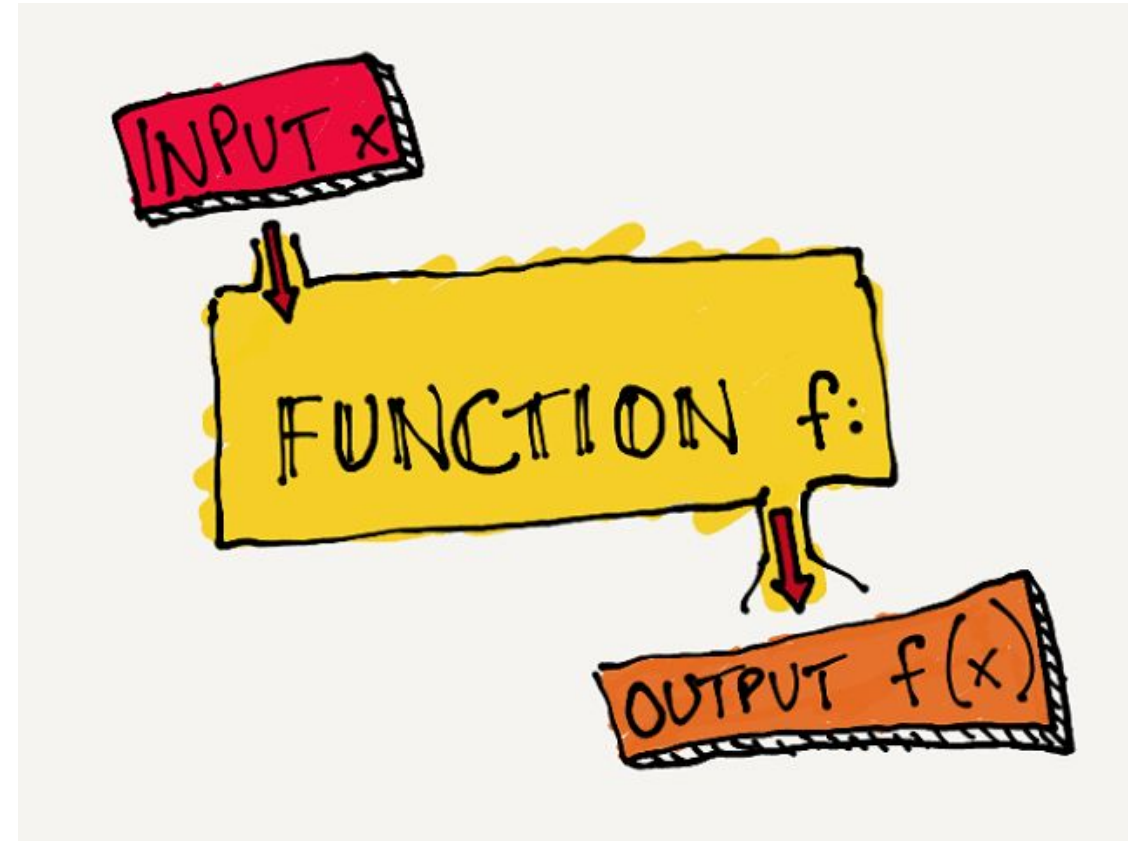
Handle exceptions gracefully and create custom error classes.

SESSION 1: FUNCTIONS

Modularizing your logic for scale

| WHY FUNCTIONS MATTER

- > **DRY Principle:** Don't Repeat Yourself. If you write the same 5 lines twice, make it a function.
- > **Abstraction:** Hide the complexity. Just call `calculate_tax()` instead of re-reading tax laws.
- > **Testability:** Small units of code are easier to verify than 500-line scripts.



| FUNCTION ANATOMY

```
def greet_user(username):  
    """Display a simple greeting."""  
    return f"Hello, {username.title()}!"  
  
# Calling the function  
message = greet_user('jesse')  
print(message)
```

The **header** starts with `def`. The **body** is indented. The `return` sends data back to the caller.

| POSITIONAL VS KEYWORD ARGS

```
def describe_pet(animal_type, pet_name):  
    print(f"I have a {animal_type} named {pet_name}.")  
  
# Positional: Order matters!  
describe_pet('hamster', 'harry')  
  
# Keyword: Order DOESN'T matter  
describe_pet(pet_name='willie', animal_type='dog')
```

| DEFAULT PARAMETERS

```
def describe_pet(pet_name, animal_type='dog'):  
    print(f"I have a {animal_type} named {pet_name}.")  
  
# Only one argument needed now  
describe_pet('willie')  
  
# Still works if we want to change it  
describe_pet('harry', 'hamster')
```

Defaults make functions **flexible** while keeping common use cases simple.

| *ARGS: ARBITRARY POSITIONALS

```
def make_pizza(size, *toppings):  
    print(f"Making a {size}-inch pizza with:")  
    for topping in toppings:  
        print(f"- {topping}")  
  
make_pizza(12, 'mushrooms', 'green peppers', 'extra cheese')
```

The * packs all extra positional arguments into a **tuple** named toppings.

| **KWARGS: ARBITRARY KEYWORDS

```
def build_profile(first, last, **user_info):  
    user_info['first_name'] = first  
    user_info['last_name'] = last  
    return user_info  
  
user = build_profile('albert', 'einstein',  
                    location='princeton', field='physics')
```

The ** packs extra keyword arguments into a **dictionary**.

LIVE DEMO: DESCRIBE PET

Let's build a "Mega Function" that combines all concepts:

```
def describe_pet(name, species="dog", *tricks, **extra_info):  
    print(f"\n{name} is a {species}")  
  
    if tricks:  
        print(f"Tricks: {' '.join(tricks)}")  
  
    if extra_info:  
        print("Details:")  
        for key, value in extra_info.items():  
            print(f"  {key}: {value}")
```

SCOPE: THE LEGB RULE

- > **L: Local** — Inside function.
- > **E: Enclosing** — In outer function.
- > **G: Global** — Module level.
- > **B: Built-in** — Python keywords.

Python searches in this **exact** order!

The screenshot shows a Python IDE window titled 'namespace.py'. The code is as follows:

```
1 x = 10
2 y = 20
3
4
5 def outer():
6     z = 30
7
8     def inner():
9         x = 30
10        print(f'x is {x}')
11        print(f'z is {z}')
12        print(f'y is {y}')
13        print(len("abc"))
14
15    inner()
16
17 outer()
18
19
```

Annotations with red arrows and text explain the LEGB rule:

- global**: Points to the global variables `x = 10` and `y = 20` at the top of the file.
- enclosed**: Points to the variable `z = 30` inside the `outer()` function, indicating it is in the enclosing scope.
- local**: Points to the variable `x = 30` inside the `inner()` function, indicating it is in the local scope.
- built-in namespace has len() function**: Points to the `len()` function call in the `inner()` function, indicating it is found in the built-in namespace.

The IDE also shows the output of the program in the 'Run' panel:

```
Run: namespace x
/Users/pankaj/Documents/PycharmProjects/PythonTutorialPro/venv/bin/python
x is 30
z is 30
y is 20
3
Process finished with exit code 0
```

| VARIABLE SHADOWING

```
x = "global"

def my_func():
    x = "local"
    print(x) # Prints "local"

my_func()
print(x) # Prints "global"
```

The local x "shadows" the global x within the function scope.

| THE `GLOBAL` KEYWORD

```
counter = 0  
  
def global_increment():  
    global counter  
    counter += 1
```

```
def local_increment():  
    counter = 0 # This is a local variable  
    counter += 1  
    return counter
```

```
local_result = local_increment()  
print(f"Local increment result: {local_result}") # 1  
  
print(f"counter: {counter}") # local does not affect the global counter  
  
global_increment()  
print(f"counter after global increment: {counter}") # 1
```

● **WARNING:** Using `global` is an anti-pattern. It makes code hard to debug. Prefer passing variables and returning results.

| DOCSTRINGS (GOOGLE STYLE)

```
def calculate_tax(amount, rate):  
    """Calculates sales tax.  
  
    Args:  
        amount (float): The total sum.  
        rate (float): Tax percentage (e.g., 0.13).  
  
    Returns:  
        float: Tax amount.  
    """  
    return amount * rate
```

| THE MUTABLE DEFAULT TRAP

```
# WRONG: List is created ONCE at definition time
```

```
def add_item(item, basket=[]):  
    basket.append(item)  
    return basket
```

```
# RIGHT: Use None as default
```

```
def add_item_fixed(item, basket=None):  
    if basket is None:  
        basket = []  
    basket.append(item)  
    return basket
```


| UNDERSTANDING THE TRAP

Why does this happen?

- > **Definition Time:** Default arguments are evaluated ONLY ONCE, when the function is defined, not when it is called.
- > **Shared Reference:** If the default is a mutable object (list, dict), every call that uses the default refers to the *same object in memory*.
- > **Persistence:** Changes made in call #1 persist into call #2.

```
def trap(val, my_list=[]):  
    my_list.append(val)  
    return my_list  
  
print(trap(1)) # [1]  
print(trap(2)) # [1, 2] <-- GHOST DATA!  
print(trap(3)) # [1, 2, 3]
```

Rule: Never use mutable types (lists, dicts, sets) as default arguments. Use None.

| COMMON FUNCTION MISTAKES

Printing vs Returning

Functions that print results can't be used in math later. Always return data.

Missing Namespaces

Trying to access local variables outside their function scope results in `NameError`.

Vague Names

`def do_stuff()` is bad. `def filter_expired_users()` is professional.

SESSION 2: LAMBDA, FILES, & EXCEPTIONS

Handling data and the "Unhappy Path"

LAMBDA EXPRESSIONS

Syntax: lambda arguments: expression

```
# Normal Function
def square(x): return x * x

# Lambda Equivalent
square = lambda x: x * x

print(square(5)) # 25
```

Best used for **one-liners** and anonymous logic transformations.

| LAMBDA WITH SORTED()

```
students = [('A', 88), ('B', 95), ('C', 72)]  
  
# Sort by score (index 1)  
sorted_students = sorted(students, key=lambda s: s[1])  
  
# Output: [('C', 72), ('A', 88), ('B', 95)]
```

Lambda provides a "key" to tell sorted() how to compare complex objects.

| LAMBDA WITH FILTER()

```
nums = [1, 2, 3, 4, 5, 6]

evens = list(filter(lambda n: n % 2 == 0, nums))

# Result: [2, 4, 6]
```

Great for quick data cleaning before processing.

| WHEN NOT TO USE LAMBDA

- If the logic requires multiple lines.
- If you need docstrings or comments inside the function.
- If you want to reuse the function in multiple modules.

"Code is read much more often than it is written." — PEP 8

Motto: Readability > Brevity.

| FILE I/O: RESOURCE SAFETY

Operating Systems have a limit on open files. If you don't **close** a file, you leak resources.

- > Files might stay locked.
- > Data might not be saved properly.
- > Your app will eventually crash.



| THE `WITH` STATEMENT

```
with open('notes.txt', 'r') as file:  
    content = file.read()  
    print(content)  
  
# File is AUTOMATICALLY closed here!
```

Always use context managers. Even if an error happens inside, Python cleans up for you.

| READING FILES SAFELY

```
with open('logs.txt') as f:  
    # Method 1: Iteration (Best for large files)  
    for line in f:  
        print(line.strip())  
  
    # Method 2: readlines() (List of lines)  
    # lines = f.readlines()
```

| WRITING & APPENDING

```
# 'w' - Write (Overwrites existing file!)  
with open('report.txt', 'w') as f:  
    f.write("Summary of Errors\n")
```

```
# 'a' - Append (Adds to end)  
with open('report.txt', 'a') as f:  
    f.write("Found 5 Error lines.")
```

| ABSOLUTE VS RELATIVE PATHS

- **Relative:** data/stats.csv (Relative to your current script folder).
- **Absolute:** C:/Users/Name/Project/data/stats.csv.

💡 **Tip:** Forward slashes (/) work on Windows, Mac, and Linux in Python!

| EXCEPTIONS: CATCHING CRASHES

Exceptions are events that interrupt the flow of a program (e.g., File Not Found, Division by Zero).

Goal: Handle them gracefully instead of letting the program die.



| TRY/EXCEPT BLOCKS

```
try:  
    age = int(input("Enter age: "))  
except ValueError:  
    print("Invalid input! Please enter a number.")
```

The try block monitors for errors. If ValueError occurs, execution jumps to except.

| HANDLING MULTIPLE ERRORS

```
try:
    with open('data.txt') as f:
        content = f.read()
        val = 10 / int(content)
except FileNotFoundError:
    print("File is missing!")
except ZeroDivisionError:
    print("Can't divide by zero!")
except Exception as e:
    print(f"Unknown error: {e}")
```

| ELSE & FINALLY

- > **else:** Runs ONLY if NO exception happened.
- > **finally:** ALWAYS runs (perfect for final cleanup).

```
try:
    f = open('test.txt')
except:
    print("Error!")
else:
    print("Success!")
finally:
    print("Closing cleanup...")
```


| CUSTOM EXCEPTIONS

```
class TooShortError (Exception):  
    pass  
  
def validate_msg (msg):  
    if len(msg) < 10:  
        raise TooShortError("Message too short!")  
  
try:  
    validate_msg("hi")  
except TooShortError as e:  
    print(e)
```

| PROJECT 3: CART ENGINE

- > Console app using **Functions** for each action.
- > Use **Lambda** for sorting/filtering items.
- > Handle "out of stock" scenarios with Exceptions.
- > Implement **fees for shipping/tax.



| PROJECT 4: LOG ANALYZER

- > Read large server.log files efficiently.
- > Count occurrences of INFO, WARN, ERROR.
- > Write an aggregated summary.txt report.
- > Gracefully handle malformed lines.



| **WARNING: TIGHT TIMELINE**

Both projects are due Wednesday Night.

- This is the most intense week of the bootcamp.
- Session 2 content is crucial for Project 4 and the Final IMS.

| FINAL PROJECT: IMS KICKOFF

Inventory & Invoice Management System

Data Persistence

Loads products from files
and saves invoices as text.

Scalable Logic

Requires structured
functions for searching
and adding stock.

Robustness

Must not crash on invalid
SKU inputs or empty data
files.

Don't start building yet. Read the spec and think about your data design.

| WHAT MAKES IMS HARD?

- › **Integration:** First project combining Weeks 1-4 logic.
- › **Zero Safety Net:** There is no in-class review; this is your final assessment.
- › **Planning:** Coding without a plan will lead to "spaghetti code." Planning is 70% of the work.

| RECAP: LOGIC & SCOPE

Functions

def, return, and flexible
`*args / **kwargs`.

Scope

LEGB rule. Local first,
Built-in last. Avoid global.

Lambda

Inline, anonymous power
for sorting and filtering.

| RECAP: FILES & ERRORS

File I/O

Always use with `open()` to prevent resource leaks.

Exceptions

Catch specific errors. Use `finally` for cleanup.

Custom Errors

Create domain-specific errors for clear logic (e.g., `InsufficientStock`).

QUESTIONS?

Check ``week-03/README.md`` for the handbook